Missing Data Mechanisms & Categorical Imputation Techniques

A practical reference guide covering MCAR, MAR, and MNAR missingness, how to recognize each, and a complete toolkit of imputation techniques for categorical data — with code, examples, and guidance on when to use which.

Contents

1. Why Missingness Mechanisms Matter

Before picking an imputation technique, it matters *why* a value is missing. The same column of missing values can call for completely different treatment depending on the underlying cause. Statisticians group missingness into three mechanisms, first formalized by Donald Rubin: **MCAR**, **MAR**, and **MNAR**. Misdiagnosing the mechanism is one of the most common silent errors in data preprocessing — it can bias a model without ever throwing an error.

> **WATCH OUT:** Getting the mechanism right comes before choosing an imputation method. A perfectly coded KNN imputer still produces a biased dataset if the missingness is actually MNAR.

2. MCAR — Missing Completely at Random

Definition: The probability of a value being missing is completely unrelated to any observed or unobserved data — including the value itself. Missingness here is pure noise; it could be swapped out for missingness in any other row and change nothing systematic about the dataset.

Example

A lab technician occasionally drops a test tube and a chemical measurement never gets recorded. The lost measurement has nothing to do with the sample's properties or with any other variable — it's an equipment accident. A survey respondent's answer is lost because of a random printing error on some forms, unrelated to who filled them out.

How to recognize it

- Missingness rate is roughly equal across all subgroups (e.g., across genders, regions, income brackets).
- A statistical test like **Little's MCAR test** fails to reject the null hypothesis (no significant pattern).
- Comparing rows with missing vs. non-missing values on other features shows no meaningful difference in distributions.

Why it matters

MCAR is the "safest" type of missingness. Since there's no hidden pattern, simple techniques (mode imputation, listwise deletion) introduce minimal bias — though you still lose statistical power if you delete rows.

```
import pandas as pd
import numpy as np

# quick visual check: compare group means/proportions
# for rows with vs without missing values in target column
missing_mask = df['col'].isna()

# does missingness relate to another numeric column?
print(df.loc[missing_mask, 'income'].mean())
print(df.loc[~missing_mask, 'income'].mean())
# similar means across the two groups --> supports MCAR
```

3. MAR — Missing at Random

Definition: The probability of a value being missing depends on *other observed variables* in the dataset, but not on the missing value itself. This is a slightly confusing name — "at random" doesn't mean "no pattern," it means the pattern is fully explainable using data you already have.

Example

In a health survey, men are less likely to report their "stress level" than women — missingness in the stress column depends on the (observed) gender column, not on the actual stress value itself. A property's garage quality is only ever missing when a different column (garage type) shows the house has no garage — the pattern is explained entirely by another observed field.

How to recognize it

- Missingness rate differs noticeably across subgroups defined by other columns.
- A logistic regression predicting "is this value missing?" from other features shows significant coefficients.
- There's a plausible, checkable relationship between missingness and another column in the data dictionary.

Why it matters

MAR is the most common real-world case, and the good news is that it's **recoverable**: because the pattern depends only on observed data, you can condition on those other columns to impute accurately — this is exactly what group-based, KNN, and model-based imputation exploit.

```
# check whether missingness in 'stress_level' depends on 'gender'
missing_rate_by_group = (
    df.assign(is_missing=df['stress_level'].isna())
      .groupby('gender')['is_missing']
      .mean()
)
print(missing_rate_by_group)
# e.g. male: 0.34, female: 0.09  --> big gap suggests MAR, driven by gender
```

4. MNAR — Missing Not at Random

Definition: The probability of a value being missing depends on the *missing value itself* (or on some unobserved variable correlated with it). This is the hardest case — the very thing you'd need to know in order to explain the missingness is the thing that's missing.

Example

People with very high incomes are less likely to disclose their income on a survey — the missingness depends on the income value itself. Patients with the most severe symptoms are more likely to drop out of a clinical trial before their final assessment is recorded — the missing outcome is related to how bad the (unrecorded) outcome actually was.

How to recognize it

- You've ruled out MCAR and MAR (no observed variable fully explains the pattern).
- Domain knowledge suggests a plausible reason tied to the value itself (e.g. sensitive/embarrassing information, extreme values, dropout tied to worsening condition).
- Statistically, MNAR can never be proven from the observed data alone — it's inferred from context, since the needed evidence is unobserved by definition.

Why it matters

MNAR is the trickiest to fix. Standard imputation methods (mode, KNN, even most model-based approaches) assume the pattern can be explained by observed data — which breaks down here. Handling MNAR properly usually requires explicit modeling of the missingness mechanism itself (e.g., selection models, pattern-mixture models), sensitivity analysis, or — most commonly in practice — treating "missing" as its own meaningful category rather than trying to guess the true value.

```
# MNAR is inferred, not proven - use domain reasoning + partial checks
# e.g., check if missingness correlates with an indirect proxy for the value

# proxy example: rows with missing 'salary' cluster in senior job titles
proxy_check = (
    df.assign(is_missing=df['salary'].isna())
      .groupby('job_title')['is_missing']
      .mean()
      .sort_values(ascending=False)
)
print(proxy_check.head())
# senior/executive titles showing much higher missing rate supports MNAR
# (higher earners more likely to withhold salary)
```

5. Comparing the Three Mechanisms

Mechanism	Depends on...	Recoverable?	Typical Example
MCAR	Nothing — pure chance	Yes, easily. Minimal bias from simple methods.	A sample was physically lost or a form was misprinted at random.
MAR	Other **observed** columns	Yes, by conditioning on the related columns.	Men skip a survey question more often than women.
MNAR	The **missing value itself** or an unobserved factor	Difficult — needs domain modeling or explicit "missing" category.	High earners decline to report income.

In practice, the three mechanisms exist on a continuum, and a single dataset can contain columns with different mechanisms. Diagnosing each column separately — rather than assuming one blanket strategy for the whole dataset — leads to more defensible results.

6. How to Diagnose Which Type You Have

There's no single test that definitively labels a column MCAR, MAR, or MNAR — but combining a few checks gets you most of the way there.

Step-by-step approach

- **Visualize missingness patterns** — use a missingness heatmap (e.g. the missingno library) to spot columns that tend to be missing together.
- **Compare subgroups** — group by other categorical columns and check whether the missing rate is roughly constant (MCAR) or varies a lot (MAR).
- **Run Little's MCAR test** — a formal statistical test; rejecting the null hypothesis rules out MCAR (but doesn't distinguish MAR from MNAR).
- **Apply domain knowledge** — ask whether it's plausible the value itself drives the missingness (sensitive data, extreme values, dropout tied to outcome). If yes, treat it as MNAR even without statistical proof.

```
# visualize missingness pattern across all columns
import missingno as msno
import matplotlib.pyplot as plt

msno.matrix(df)
plt.show()

msno.heatmap(df)   # shows correlation between which columns are missing together
plt.show()
```

WATCH OUT: MNAR can't be confirmed purely from the data you have — by definition, the evidence needed lives in the unobserved value. Treat MNAR conclusions as a reasoned judgment call, not a statistical certainty.

7. Categorical Imputation Techniques

Once you understand *why* values are missing, choose *how* to fill them. Below is the full toolkit for categorical columns, each with a code example.

7.1 Mode Imputation

Best fits: MCAR

Replace missing values with the single most frequent category in the column.

```
df['col'] = df['col'].fillna(df['col'].mode()[0])
```

Use when: Missing rate is small (roughly under 5%) and appears random, or one category strongly dominates.

Avoid when: The column is fairly balanced across categories — this would artificially inflate one category and distort the distribution.

7.2 Constant / Placeholder Imputation

Best fits: MNAR (missingness itself is meaningful)

Fill with a brand-new category such as "None" or "Missing", treating the absence of a value as a valid state rather than an unknown one.

```
df['garage_qual'] = df['garage_qual'].fillna('None')
```

Use when: The NaN represents a real condition (e.g. the feature genuinely doesn't exist for that row) rather than lost/unknown data.

Avoid when: Missingness is truly unexplained/random — adding a fake category then just adds noise.

7.3 Domain-Rule / Logical Imputation

Best fits: MAR (explainable by another observed column)

Derive the correct value directly from a known relationship with another column, instead of estimating it statistically.

```
# if 'garage_type' is missing, related garage columns are logically 'None' too
related_cols = ['garage_qual', 'garage_cond', 'garage_finish']
df.loc[df['garage_type'].isna(), related_cols] = 'None'
```

Use when: You can prove/derive the true value with certainty from another field — always the top choice when applicable.

Avoid when: No such deterministic relationship exists between columns.

7.4 Group-Based / Stratified Mode Imputation

Best fits: MAR

Instead of a single global mode, impute using the most frequent category *within a relevant subgroup* (e.g. per region, per class).

```
df['col'] = (
    df.groupby('neighborhood')['col']
      .transform(lambda x: x.fillna(x.mode()[0] if not x.mode().empty else 'None'))
)
```

Use when: The missing value likely depends on another categorical grouping variable, and mode-per-group beats a single overall mode.

Avoid when: Groups are too small (mode becomes unstable/noisy) or the grouping variable has no real relationship to the column.

7.5 Hot-Deck Imputation

Best fits: MAR

Fill a missing value by copying it from a similar ("donor") row elsewhere in the same dataset, matched on shared characteristics.

```
def hot_deck_impute(df, target_col, match_cols):
    df = df.copy()
    missing_idx = df[df[target_col].isna()].index
    for idx in missing_idx:
        match_values = df.loc[idx, match_cols]
        donors = df.dropna(subset=[target_col])
        for col in match_cols:
            donors = donors[donors[col] == match_values[col]]
        if not donors.empty:
            df.loc[idx, target_col] = donors[target_col].sample(1).values[0]
    return df

df = hot_deck_impute(df, target_col='col', match_cols=['neighborhood', 'house_style
'])
```

Use when: You want realistic, dataset-native values rather than a computed statistic — common in survey-style data.

Avoid when: There's no reliable way to define a 'similar' row, or the dataset is too small to have good donor matches.

7.6 Cold-Deck Imputation

Best fits: MAR / MNAR (with external reference)

Same idea as hot-deck, but donor values come from a trusted *external* dataset rather than the same one.

```
# external_df has the same key + target column from a trusted reference source
df = df.merge(
    external_df[['property_id', 'col']],
    on='property_id', how='left', suffixes=('', '_ext')
)
df['col'] = df['col'].fillna(df['col_ext'])
df = df.drop(columns=['col_ext'])
```

Use when: A trustworthy external dataset with matching keys and the same variable is available.

Avoid when: No reliable external source exists, or its collection methodology isn't compatible with your data.

7.7 Forward Fill / Backward Fill

Best fits: MCAR / MAR (ordered data only)

Carries the previous (ffill) or next (bfill) row's value forward/backward — only meaningful when row order is significant.

```
df_sorted = df.sort_values('timestamp')
df_sorted['status'] = df_sorted['status'].ffill()   # or .bfill()
```

Use when: Data is sequential/ordered (time series, logs) and neighboring rows are likely to share the same category.

Avoid when: Row order carries no real meaning, as in typical unordered tabular datasets.

7.8 Random Sample Imputation

Best fits: MCAR

Fill missing values by randomly sampling from the column's own existing distribution, preserving its overall shape.

```
import numpy as np
np.random.seed(42)   # for reproducibility

observed = df['col'].dropna().values
missing_mask = df['col'].isna()
df.loc[missing_mask, 'col'] = np.random.choice(observed, size=missing_mask.sum())
```

Use when: You want to preserve the natural variance/shape of the distribution, e.g. in exploratory analysis.

Avoid when: Reproducibility matters and a seed can't be fixed, or you're in a production setting requiring consistent output.

7.9 K-Nearest Neighbors (KNN) Imputation

Best fits: MAR

Encodes categories numerically, then imputes using the most common category among the K most similar rows (by other features).

```
from sklearn.impute import KNNImputer
from sklearn.preprocessing import OrdinalEncoder
import numpy as np

encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
df[['col_enc']] = encoder.fit_transform(df[['col']].astype(str))
df.loc[df['col'].isna(), 'col_enc'] = np.nan

imputer = KNNImputer(n_neighbors=5)
df['col_enc'] = imputer.fit_transform(df[['col_enc']]).round()
df['col'] = encoder.inverse_transform(df[['col_enc']])
```

Use when: Missingness likely correlates with other features, and dataset size is moderate (not huge).

Avoid when: Dataset is very large (expensive distance computations) or has too many irrelevant/noisy features.

7.10 Model-Based / Predictive Imputation

Best fits: MAR (can partially address MNAR with proxies)

Train a classifier using other columns as features to directly predict the missing category.

```
from sklearn.ensemble import RandomForestClassifier

known = df[df['col'].notna()]
unknown = df[df['col'].isna()]

feature_cols = ['neighborhood', 'house_style', 'year_built']  # already-encoded fea
tures
X_train, y_train = known[feature_cols], known['col']
X_pred = unknown[feature_cols]

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
df.loc[df['col'].isna(), 'col'] = model.predict(X_pred)
```

Use when: Accuracy matters a lot (production models, competitions), and there's enough correlated data to train a decent predictor.

Avoid when: Dataset is small (overfitting risk), or the added complexity isn't justified by the accuracy gain.

7.11 Multiple Imputation (MICE)

Best fits: MAR

Generates several plausible imputed datasets (capturing uncertainty), analyzes each, then pools the results — the statistically rigorous option.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.preprocessing import OrdinalEncoder
import numpy as np

encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
encoded = encoder.fit_transform(df[['col1', 'col2', 'col3']].astype(str))

imputer = IterativeImputer(max_iter=10, random_state=42)
imputed = imputer.fit_transform(encoded)

df[['col1', 'col2', 'col3']] = encoder.inverse_transform(np.round(imputed))
```

Use when: Statistical rigor and uncertainty quantification matter (common in research), and missingness is substantial.

Avoid when: You just need a quick, practical fill for an ML pipeline — MICE is heavier to implement and interpret than single imputation.

7.12 Missing Indicator + Any Imputation

Best fits: MNAR (flags informative missingness)

Alongside filling the value with any method above, add a binary column flagging which rows were originally missing — useful when missingness itself may be predictive.

```
df['col_was_missing'] = df['col'].isna().astype(int)
df['col'] = df['col'].fillna(df['col'].mode()[0])
```

Use when: Missingness itself might correlate with the target variable (common under MNAR) — the model can then learn from the fact of missingness, not just the filled value.

Avoid when: Missingness is truly random and carries no signal — the flag just adds noise/dimensionality.

7.13 Deep Learning-Based Imputation (Autoencoders / GAIN)

Best fits: MAR / complex MNAR

Neural network architectures (denoising autoencoders, Generative Adversarial Imputation Networks) learn complex, non-linear missingness patterns.

```
# conceptual outline (requires a GAIN or autoencoder implementation)
# 1. Encode categorical columns numerically
# 2. Train an autoencoder to reconstruct rows, treating missing entries as
#    withheld during training so the network learns to reconstruct them
# 3. Use the trained network to predict missing categorical entries

from sklearn.preprocessing import OrdinalEncoder
# ... encode df, mask missing values, feed into a GAIN/autoencoder model
# (see the `datawig` or custom PyTorch/TensorFlow GAIN implementations)
```

Use when: Dataset is large, missingness patterns are complex/non-linear, and the infrastructure/expertise to train and validate deep models is available.

Avoid when: Dataset is small-to-medium — deep learning is overkill and prone to instability with limited data.

8. Decision Guide — Which Technique, When

The table below maps each missingness mechanism to the techniques best suited for it, in priority order — start at the top of each row and move down only if the condition doesn't hold.

If missingness is...	Prefer, in order
MCAR (pure chance)	1. Mode imputation (small % missing) 2. Random sample imputation (preserve distribution) 3. Simple deletion, if row loss is negligible
MAR (explained by other observed columns)	1. Domain-rule imputation (if a certain rule exists) 2. Group-based mode / hot-deck 3. KNN imputation 4. Model-based (Random Forest, etc.) 5. Multiple Imputation (MICE), for statistical rigor
MNAR (depends on the missing value itself)	1. Constant/placeholder category ("None"/"Missing") 2. Missing indicator column + any base imputation 3. Domain-informed modeling of the missingness mechanism 4. Deep learning methods, for large/complex data

General priority order (default reasoning)

- **Is there a logical/derivable reason for the missing value?** → Domain-rule imputation — most reliable, since it's not a guess.
- **Is the missingness itself meaningful (a valid state, not "unknown")?** → Constant/placeholder imputation.
- **Is it a handful of missing rows with no discernible pattern?** → Mode imputation — simplest, safest default.
- **Is missingness spread out and possibly tied to other features, with enough data?** → KNN or model-based, depending on accuracy needs and compute budget.
- **Is the data ordered/sequential?** → Forward/backward fill.
- **Want to preserve natural variance in exploratory work?** → Random sample imputation.
- **Could missingness itself be predictive of the outcome?** → Add a missing-indicator column regardless of which base method you use to fill the value.

> **TIP:** The overarching principle: prefer the technique that uses the most certain information first (logical rules > meaningful placeholders), and only escalate to statistical or ML-based imputation when you genuinely can't determine the value another way.

End of guide.